

Lab1: Temporal Difference Learning

Lab Objective:

In this lab, you will learn temporal difference learning (TD) algorithm by solving the 2048 game using an n -tuple network. You will implement two variants, TD-state and TD-after-state, and compare them.

Important Date:

1. Submission Deadline: 9/27 (Sun) 23:59

2. Demo date: 10/06 (Tue)

Turn in:

1. Experiment report (.pdf)

2. Source code (.cpp) [NOT including model weights]. You must submit two files:

- `td_state.cpp`: the TD-state implementation.
- `td_afterstate.cpp`: the TD-after-state implementation.

Notice: zip all files with name “**RL_LAB1_StudentId_Name.zip**”,

e.g.: 「RL_LAB1_313551088_楊右宇.zip」

Lab Description:

- Understand the concept of (before-)state and after-state.
- Learn to construct and design an n -tuple network.
- Understand TD algorithm.
- Compare learning $V(\text{state})$ and learning $V(\text{after-state})$.
- Understand Q-learning network training.

Requirements:

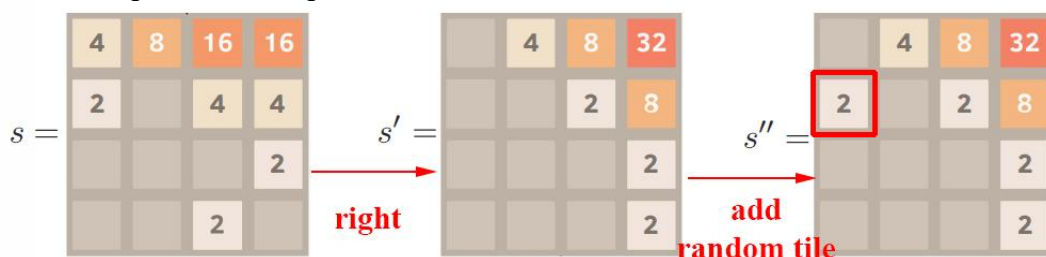
- Implement TD(0) algorithm
 - Construct an n -tuple network.
 - Action selection according to the n -tuple network.
 - Implement **both** TD-state and TD-after-state:
 - ◆ **TD-state**: learn $V(s)$. Action selection uses the expected value over all possible next states s' . You must use $P(\text{popup tile } 2) = 0.7$ and $P(\text{popup tile } 4) = 0.3$.
 - ◆ **TD-after-state**: learn $V(s')$. Action selection uses $r + V(s')$. No expectation over popup tiles is needed.
 - For each variant, calculate the TD-target and the TD-error.
 - Understand temporal difference learning mechanisms.

Game Environment – 2048:

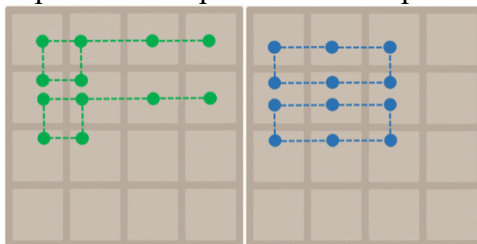
- Introduction: 2048 is a single-player sliding block puzzle game. The game's objective is to slide numbered tiles on a grid to combine them to create a tile with the number 2048.
- Actions: **Up, Down, Left, Right**
- Reward: The score is the value of new tile when two tiles are combined.
- Random tile: after each move, a new tile appears in a random empty cell. $P(\text{popup tile } 2) = 0.7$ and $P(\text{popup tile } 4) = 0.3$. This applies to both TD-state and TD-after-state.

		2	4
		4	8
	2	16	32
	2	2	16

- A sample of two-step state transition

Implementation Details:**Network Architecture**

- n -tuple patterns: 4×6 -tuples with all possible isomorphisms

**Training Arguments**

- Learning rate: 0.1
 - Learning rate for features of n -tuple network with m features: $0.1 \div m$
- Train each variant for at least 100k episodes.

Algorithm:**A pseudocode of the game engine and training.** (modified backward training method)

```

function PLAY GAME
     $score \leftarrow 0$ 
     $s \leftarrow \text{INITIALIZE GAME STATE}$ 
    while IS NOT TERMINAL STATE( $s$ ) do
         $a \leftarrow \text{argmax}_{a' \in A(s)} \text{EVALUATE}(s, a')$ 
         $r, s', s'' \leftarrow \text{MAKE MOVE}(s, a)$ 
        SAVE RECORD( $s, a, r, s', s''$ )
         $score \leftarrow score + r$ 
         $s \leftarrow s''$ 
    for ( $s, a, r, s', s''$ ) FROM TERMINAL DOWNT0 INITIAL do
        LEARN EVALUATION( $s, a, r, s', s''$ )
    return  $score$ 

function MAKE MOVE( $s, a$ )
     $s', r \leftarrow \text{COMPUTE AFTERSTATE}(s, a)$ 
     $s'' \leftarrow \text{ADD RANDOM TILE}(s')$ 
    return ( $r, s', s''$ )

```

TD-state

```

function EVALUATE( $s, a$ )
     $s', r \leftarrow \text{COMPUTE AFTERSTATE}(s, a)$ 
     $S'' \leftarrow \text{ALL POSSIBLE NEXT STATES}(s')$ 
    return  $r + \sum_{s'' \in S''} P(s, a, s'') V(s'')$ 

function LEARN EVALUATION( $s, a, r, s', s''$ )
     $V(s) \leftarrow V(s) + \alpha(r + V(s'') - V(s))$ 

```

TD-after-state

```

function EVALUATE( $s, a$ )
     $s', r \leftarrow \text{COMPUTE AFTERSTATE}(s, a)$ 
    return  $r + V(s')$ 

function LEARN EVALUATION( $s, a, r, s', s''$ )
     $a_{\text{next}} \leftarrow \text{argmax}_{a' \in A(s'')} \text{EVALUATE}(s'', a')$ 
     $s'_{\text{next}}, r_{\text{next}} \leftarrow \text{COMPUTE AFTERSTATE}(s'', a_{\text{next}})$ 
     $V(s') \leftarrow V(s') + \alpha(r_{\text{next}} + V(s'_{\text{next}}) - V(s'))$ 

```

Note: $P(s, a, s'') = 0.7 / k$ for a new tile 2 and $0.3 / k$ for a new tile 4, where k is the number of empty cells in s' .

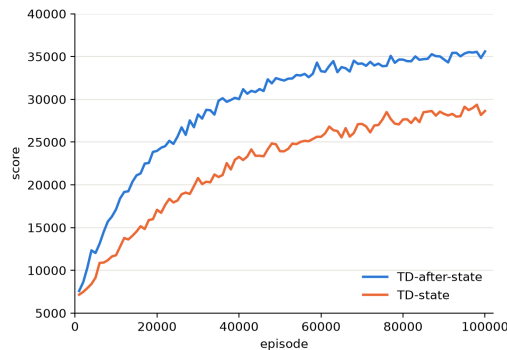
Note: If s'' is a terminal state, $V(s'') = 0$. In TD-after-state, $A(s'')$ is empty, so the TD-target is 0.

Scoring Criteria:

Show your work, otherwise no credit will be granted.

- Report (30%)
 - Plots of mean scores for both variants, at least 100k episodes each. One figure with two curves is acceptable. (10%)

E.g.



(Example format only; not the expected result.)

- Describe the implementation and the usage of n -tuple network. (5%)
 - Explain the mechanism of TD(0), and the difference between TD-state and TD-after-state. (5%)
 - Describe your implementation of both variants, with action selection and a TD-backup diagram for each. (10%)
- Demo (70%)
 - The 2048-tile win rate in 1000 games, $[winrate_{2048}]$. (50%)
 - ◆ You can choose which model to use in the demo: TD-state or TD-after-state.
 - ◆ Use the model that gave your best result.
 - ◆ The weights must be trained by your submitted `td_state.cpp` or `td_afterstate.cpp`.

E.g.

1000	mean = 21355.2	max = 64492
128	100%	(0.1%)
256	99.9%	(1.4%)
512	98.5%	(11.6%)
1024	86.9%	(51.2%)
2048	35.7%	(34.6%)
4096	1.1%	(1.1%)

- Questions. (20%)

References:

- [1] Szubert, Marcin, and Wojciech Jaśkowski. "Temporal difference learning of N-tuple networks for the game 2048." 2014 IEEE Conference on Computational Intelligence and Games. IEEE, 2014.
- [2] Kun-Hao Yeh, I-Chen Wu, Chu-Hsuan Hsueh, Chia-Chuan Chang, Chao-Chin Liang, and Han Chiang, Multi-Stage Temporal Difference Learning for 2048-like Games, accepted by IEEE Transactions on Computational Intelligence and AI in Games (SCI), doi: 10.1109/TCIAIG.2016.2593710, 2016.
- [3] Oka, Kazuto, and Kiminori Matsuzaki. "Systematic selection of n-tuple networks for 2048." International Conference on Computers and Games. Springer International Publishing, 2016.
- [4] moporgic. "Basic implementation of 2048 in Python." Retrieved from Github: <https://github.com/moporgic/2048-Demo-Python>.
- [5] moporgic. "Temporal Difference Learning for Game 2048 (Demo)." Retrieved from Github: <https://github.com/moporgic/TDL2048-Demo>.
- [6] lukewayne123. "2048-Framework" Retrieved from Github: <https://github.com/lukewayne123/2048-Framework>.